Лабораторийн ажил №3

Сэдэв: Process Control Block (PCB) ба процессын симуляци

Хичээл: Үйлдлийн систем

Гүйцэтгэсэн: Н.Мөнхпүрэв, 23B1NUM0114

Огноо: 2026-10-05

1. ОРШИЛ

Энэ лабораторийн ажилд үйлдлийн системийн процесс удирдлагын үндсэн өгөгдлийн бүтэц болох Process Control Block буюу PCB-г C хэл дээр энгийн байдлаар загварчилж хийв. Ажлыг гурван хэсэгт хийсэн: (1) random массив үүсгэж эрэмбэлэх Process1.c, (2) хоёр матриц нэмж үр дүн гаргах Process2.c, (3) тэмдэгт мөрийн давтамж тоолох Process3.c.

Эдгээр гурван процессыг тус тусад нь ажиллуулах боломжтойгоос гадна Lab3.c файлд нэгтгэж, round-robin simulation хэлбэрээр ээлжлэн ажиллуулсан. Энэ нь бодит kernel түвшний context switching хийхгүй боловч процессын төлөв, тоолуур, PCB мэдээллийг хэрхэн шинэчилж болохыг харуулна.

2. ЗОРИЛГО

PCB бүтэц ашиглан процессын process_id, priority, state, process_counter зэрэг мэдээллийг хадгалж, процессуудыг ээлжлэн ажиллуулах энгийн simulation хийх.

Зорилтууд:

1) struct PCB бүтцийг тодорхойлж процессын мэдээллийг хадгалах.

2) Process1.c, Process2.c, Process3.c гэсэн гурван өөр логиктой програм бичих.

3) Процесс бүрийн state болон process_counter утгыг давталт бүрт шинэчлэх.

4) Lab3.c файлд гурван процессыг нэгтгэж round-robin simulation хийх.

5) Makefile ашиглан хөрвүүлж, гаралтыг шалгах.

3. ОНОЛЫН СУДАЛГАА

3.1 Онол

Процесс гэдэг нь ажиллаж байгаа програм бөгөөд зөвхөн эх код биш, харин тухайн агшинд ямар төлөвтэй, ямар нөөц ашиглаж байгаа, дараагийн ямар зааврыг биелүүлэх зэрэг удирдлагын мэдээлэлтэй байдаг. Үйлдлийн систем энэ мэдээллийг PCB буюу Process Control Block-д хадгалдаг [1].

PCB-д ихэвчлэн process id, process state, program counter, CPU register-ийн мэдээлэл, scheduling-тэй холбоотой priority зэрэг талбарууд ордог [2]. Энэ лабораторийн хувьд ойлгоход хялбар болгохын тулд process_id, priority, state, process_counter гэсэн дөрвөн талбар ашигласан.

Процессын төлөв нь READY, RUNNING, WAITING гэх мэт утгатай байж болно. READY нь ажиллахад бэлэн, RUNNING нь яг одоо ажиллаж байгаа, WAITING нь тодорхой үйл явдал эсвэл хугацаа хүлээж байгаа гэсэн утгатай. Програмд эдгээрийг 0, 1, 2 гэсэн бүхэл тоогоор илэрхийлсэн.

Round-robin scheduling нь процесс бүрт ээлжээр ажиллах боломж олгодог энгийн scheduling арга юм [3]. Энэ ажлын Lab3.c файлд бодит time quantum биш, харин while(1) давталт дотор гурван процессын нэг алхмыг дараалаар дуудсан.

3.2 Судалгаа

Асуулт: нэгтгэх програмыг цэстэй байдлаар хийх үү эсвэл гурван процессыг ээлжлэн ажиллуулах уу? Даалгаварт хоёр хувилбар байсан боловч scheduling ойлголтыг харуулахын тулд round-robin хувилбарыг сонгов.

Хүснэгт 1. Нэгтгэх хувилбаруудын харьцуулалт

Шалгуур	Menu хувилбар	Round-robin хувилбар
Ажиллах зарчим	Хэрэглэгч нэг процесс сонгоно.	Гурван процесс дараалаар ажиллана.
Scheduling ойлголт	Бага харагдана.	Ээлжлэн ажиллах санаа тод харагдана.
Энэ ажилд	Тус тусад нь шалгахад тохиромжтой.	Нэгдсэн simulation-д илүү тохиромжтой.

Хүснэгт 1-ээс харахад энэ лабораторийн зорилгод round-robin хувилбар илүү тохиромжтой. Иймээс process1_step(), process2_step(), process3_step() функцуудыг дараалаар нь дуудаж, процесс бүрийн counter-ийг тус тусад нь нэмэгдүүлэх шийдэл авсан.

4. ХЭРЭГЖҮҮЛЭЛТ

Програмыг таван үндсэн файлаар хийсэн: Process1.c, Process2.c, Process3.c, Lab3.c, Makefile. Бүрэн эх кодыг хавсралтад оруулав.

4.1 Шийдэл ба кодын тайлбар

Оролт: random тоо үүсгэхээс бусад хэрэглэгчийн оролт байхгүй. Гаралт: массив, матриц, тэмдэгтийн давтамж болон процесс бүрийн PCB мэдээлэл.

I хэсэг. PCB бүтэц

PCB бүтэц нь процессын удирдлагад хэрэгтэй дөрвөн утгыг хадгална.

Код 1. PCB бүтцийн тодорхойлолт

```
#define READY 0
#define RUNNING 1
#define WAITING 2

struct PCB {
    int process_id;
    int priority;
    int state;
    int process_counter;
};
```

II хэсэг. Process1.c

Process1 нь 10 ширхэг random тоо үүсгээд bubble sort аргаар өсөхөөр эрэмбэлнэ. Давталт бүрт process_counter нэгээр нэмэгдэж, PCB мэдээлэл хэвлэгдэнэ.

Код 2. Process1-ийн үндсэн ажиллагаа

```
for (i = 0; i < n; i++) {
    numbers[i] = rand() % 100;
}

sort_array(numbers, n);
print_pcb(pcb);
```

III хэсэг. Process2.c

Process2 нь A болон B гэсэн 3x3 хэмжээтэй хоёр матриц үүсгэж, ижил байрлалын элементүүдийг нэмэн C матрицад хадгална.

Код 3. Матриц нэмэх хэсэг

```
for (i = 0; i < m; i++) {
    for (j = 0; j < n; j++) {
        a[i][j] = rand() % 10;
        b[i][j] = rand() % 10;
        c[i][j] = a[i][j] + b[i][j];
    }
}
```

IV хэсэг. Process3.c

Process3 нь operating system гэсэн тэмдэгт мөрийн тэмдэг бүрийн давтамжийг count[256] массив ашиглан тоолдог.

Код 4. Давтамж тоолох хэсэг

```
for (i = 0; i < 256; i++) {
    count[i] = 0;
}
```

```
for (i = 0; text[i] != '\0'; i++) {
    count[(unsigned char)text[i]]++;
}
```

V хэсэг. Lab3.c

Lab3.c файлд гурван процессын логикийг нэг алхам гүйцэтгэх функц хэлбэрээр бичиж, main() функцэд round-robin маягаар дуудсан.

Код 5. Round-robin simulation-ийн үндсэн давталт

```
while (1) {
    process1_step(&p1);
    process2_step(&p2);
    process3_step(&p3);

    printf("\nНэг round дууслаа.\n");
    sleep(1);
}
```

4.2 Туршилт ба үр дүн

Ажиллуулах орчин: Linux, gcc, make. Хөрвүүлэхэд gcc-ийн -Wall -Wextra -std=c11 тохиргоо ашигласан.

Код 6. Туршилтын командууд

```
make
./lab3
./process1
./process2
./process3
make clean
```

Хүснэгт 2. Туршилтын үр дүн

№	Үйлдэл	Гарсан үр дүн
1	make	lab3, process1, process2, process3 файлууд алдаагүй үүссэн.
2	./lab3	Process1, Process2, Process3 дарааллаар ажилласан.
3	Process1	Random массив өсөхөөр эрэмбэлэгдсэн.
4	Process2	C матриц нь A ба B матрицын нийлбэр болсон.
5	Process3	operating system мөрийн тэмдэгтийн давтамж зөв гарсан.
6	Counter	Round бүрт процесс бүрийн counter нэгээр нэмэгдсэн.

Нэгдсэн програмын нэг round-ийн жишээ гаралт:

Гаралт 1. ./lab3 програмын жишээ

```
Lab 3 - PCB ба round-robin simulation
Програмыг зогсоох бол CTRL + C дарна.

--- Process 1: Random array sort ---
Random array: 1 10 99 83 77 69 46 22 62 19
Sorted array: 1 10 19 22 46 62 69 77 83 99
PCB: PID=1, Priority=3, State=1, Counter=1

--- Process 2: Matrix addition ---
C = A + B:
   9   5   8
   6   0   9
  17   8  12
PCB: PID=2, Priority=2, State=1, Counter=1

--- Process 3: Character frequency ---
String: "operating system"
'space': 1
'e': 2
's': 2
't': 2
PCB: PID=3, Priority=1, State=1, Counter=1
```

Үр дүнгийн тайлбар: бүх процесс дарааллаар ажиллаж, counter утга round бүрт 1, 2, 3 гэж өссөн. Иймээс PCB мэдээлэл шинэчлэгдэж, round-robin simulation зөв ажиллаж байна гэж үзэв.

5. ДҮГНЭЛТ

Энэ лабораторийн ажлаар PCB нь процессын төлөв болон удирдлагын мэдээллийг хадгалах гол бүтэц болохыг ойлголоо. C хэлний struct ашиглан процессын ID, priority, state, counter утгуудыг хадгалж, ажиллах явцад шинэчилж болохыг туршсан.

Round-robin simulation хийснээр процессууд нэг систем дотор ээлжлэн ажиллаж байгаа санааг энгийн програмын түвшинд харуулж чадсан. Энэ нь бодит kernel scheduling биш боловч CPU scheduling болон context switching сэдвийг ойлгоход суурь ойлголт өгч байна.

6. ХЯЗГААРЛАЛТ БА САЙЖРУУЛАХ БОЛОМЖ

Энэ програм бодит үйлдлийн системийн процесс үүсгэхгүй. fork(), exec(), wait() зэрэг системийн дуудлага ашиглаагүй тул context switching бодитоор хийгдэхгүй. Мөн priority талбарыг scheduling шийдвэрт ашиглаагүй.

Цаашид priority-based scheduling нэмж, процесс бүрт өөр өөр time quantum өгөх, эсвэл pthread ашиглан зэрэгцээ ажиллуулах байдлаар сайжруулж болно.

7. АШИГЛАСАН МАТЕРИАЛ

1) Лабораторийн ажил 3. Process Control Block (PCB) ба процессын симуляци, хичээлийн даалгаврын PDF.

2) Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 10th Edition, Chapter 3: Processes.

3) Operating System Concepts, 10th Edition, Chapter 5: CPU Scheduling.

4) Linux manual pages: gcc, make, sleep.

Хавсралт А. Эх код

Лабораторийн үндсэн эх кодуудыг мөрийн дугаартайгаар хавсаргав.

Lab3.c

```
 1  #include <stdio.h>
 2  #include <stdlib.h>
 3  #include <string.h>
 4  #include <time.h>
 5  #include <unistd.h>
 6
 7  #define READY 0
 8  #define RUNNING 1
 9  #define WAITING 2
10
11  struct PCB {
12      int process_id;
13      int priority;
14      int state;
15      int process_counter;
16  };
17
18  void print_pcb(struct PCB pcb)
19  {
20      printf("PCB: PID=%d, Priority=%d, State=%d, Counter=%d\n",
21             pcb.process_id, pcb.priority, pcb.state, pcb.process_counter);
22  }
23
24  void print_array(int a[], int n)
25  {
26      int i;
27
28      for (i = 0; i < n; i++) {
29          printf("%d ", a[i]);
30      }
31      printf("\n");
32  }
33
34  void sort_array(int a[], int n)
35  {
36      int i, j, temp;
37
38      for (i = 0; i < n - 1; i++) {
39          for (j = 0; j < n - i - 1; j++) {
40              if (a[j] > a[j + 1]) {
41                  temp = a[j];
42                  a[j] = a[j + 1];
43                  a[j + 1] = temp;
44              }
45          }
46      }
47  }
48
49  void process1_step(struct PCB *pcb)
50  {
51      int numbers[10];
52      int n = 10;
53      int i;
54
55      pcb->state = RUNNING;
56      pcb->process_counter++;
57
58      printf("\n--- Process 1: Random array sort ---\n");
```

```
    for (i = 0; i < n; i++) {
        numbers[i] = rand() % 100;
    }

    printf("Random array: ");
    print_array(numbers, n);
    sort_array(numbers, n);
    printf("Sorted array: ");
    print_array(numbers, n);
    print_pcb(*pcb);

    pcb->state = READY;
}

void print_matrix(int matrix[3][3], int m, int n)
{
    int i, j;

    for (i = 0; i < m; i++) {
        for (j = 0; j < n; j++) {
            printf("%4d", matrix[i][j]);
        }
        printf("\n");
    }
}

void process2_step(struct PCB *pcb)
{
    int a[3][3], b[3][3], c[3][3];
    int m = 3, n = 3;
    int i, j;

    pcb->state = RUNNING;
    pcb->process_counter++;

    printf("\n--- Process 2: Matrix addition ---\n");
    for (i = 0; i < m; i++) {
        for (j = 0; j < n; j++) {
            a[i][j] = rand() % 10;
            b[i][j] = rand() % 10;
            c[i][j] = a[i][j] + b[i][j];
        }
    }

    printf("A matrix:\n");
    print_matrix(a, m, n);
    printf("B matrix:\n");
    print_matrix(b, m, n);
    printf("C = A + B:\n");
    print_matrix(c, m, n);
    print_pcb(*pcb);

    pcb->state = READY;
}

void process3_step(struct PCB *pcb)
{
    char text[] = "operating system";
    int count[256];
    int i;

    pcb->state = RUNNING;
    pcb->process_counter++;

    for (i = 0; i < 256; i++) {
```

```
124         count[i] = 0;
125     }
126
127     for (i = 0; text[i] != '\0'; i++) {
128         count[(unsigned char)text[i]]++;
129     }
130
131     printf("\n--- Process 3: Character frequency ---\n");
132     printf("String: \"%s\"\n", text);
133     printf("Frequency:\n");
134     for (i = 0; i < 256; i++) {
135         if (count[i] > 0) {
136             if (i == ' ') {
137                 printf("'space': %d\n", count[i]);
138             } else {
139                 printf("'%c': %d\n", i, count[i]);
140             }
141         }
142     }
143     print_pcb(*pcb);
144
145     pcb->state = READY;
146 }
147
148 int main(void)
149 {
150     struct PCB p1 = {1, 3, READY, 0};
151     struct PCB p2 = {2, 2, READY, 0};
152     struct PCB p3 = {3, 1, READY, 0};
153
154     srand(time(NULL));
155
156     printf("Lab 3 - PCB ба round-robin simulation\n");
157     printf("Програмыг зогсоох бол CTRL + C дарна.\n");
158
159     while (1) {
160         process1_step(&p1);
161         process2_step(&p2);
162         process3_step(&p3);
163
164         printf("\nНэг round дууслаа.\n");
165         sleep(1);
166     }
167
168     return 0;
169 }
```

Process1.c

```
 1 #include <stdio.h>
 2 #include <stdlib.h>
 3 #include <time.h>
 4 #include <unistd.h>
 5
 6 #define READY 0
 7 #define RUNNING 1
 8 #define WAITING 2
 9
10 struct PCB {
11     int process_id;
12     int priority;
13     int state;
14     int process_counter;
15 };
16
17 void print_pcb(struct PCB pcb)
```

```
18  {
19      printf("PCB: PID=%d, Priority=%d, State=%d, Counter=%d\n",
20             pcb.process_id, pcb.priority, pcb.state, pcb.process_counter);
21  }
22
23  void print_array(int a[], int n)
24  {
25      int i;
26
27      for (i = 0; i < n; i++) {
28          printf("%d ", a[i]);
29      }
30      printf("\n");
31  }
32
33  void sort_array(int a[], int n)
34  {
35      int i, j, temp;
36
37      for (i = 0; i < n - 1; i++) {
38          for (j = 0; j < n - i - 1; j++) {
39              if (a[j] > a[j + 1]) {
40                  temp = a[j];
41                  a[j] = a[j + 1];
42                  a[j + 1] = temp;
43              }
44          }
45      }
46  }
47
48  int main(void)
49  {
50      struct PCB pcb = {1, 3, READY, 0};
51      int n = 10;
52      int numbers[10];
53      int i;
54
55      srand(time(NULL));
56
57      while (1) {
58          pcb.state = RUNNING;
59          pcb.process_counter++;
60
61          printf("\n--- Process 1: Random array sort ---\n");
62          for (i = 0; i < n; i++) {
63              numbers[i] = rand() % 100;
64          }
65
66          printf("Random array: ");
67          print_array(numbers, n);
68
69          sort_array(numbers, n);
70
71          printf("Sorted array: ");
72          print_array(numbers, n);
73          print_pcb(pcb);
74
75          pcb.state = WAITING;
76          sleep(1);
77      }
78
79      return 0;
80  }
```

Process2.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>

#define READY 0
#define RUNNING 1
#define WAITING 2

struct PCB {
    int process_id;
    int priority;
    int state;
    int process_counter;
};

void print_pcb(struct PCB pcb)
{
    printf("PCB: PID=%d, Priority=%d, State=%d, Counter=%d\n",
           pcb.process_id, pcb.priority, pcb.state, pcb.process_counter);
}

void print_matrix(int matrix[3][3], int m, int n)
{
    int i, j;

    for (i = 0; i < m; i++) {
        for (j = 0; j < n; j++) {
            printf("%4d", matrix[i][j]);
        }
        printf("\n");
    }
}

int main(void)
{
    struct PCB pcb = {2, 2, READY, 0};
    int m = 3, n = 3;
    int a[3][3], b[3][3], c[3][3];
    int i, j;

    srand(time(NULL));

    while (1) {
        pcb.state = RUNNING;
        pcb.process_counter++;

        printf("\n--- Process 2: Matrix addition ---\n");
        for (i = 0; i < m; i++) {
            for (j = 0; j < n; j++) {
                a[i][j] = rand() % 10;
                b[i][j] = rand() % 10;
                c[i][j] = a[i][j] + b[i][j];
            }
        }

        printf("A matrix:\n");
        print_matrix(a, m, n);
        printf("B matrix:\n");
        print_matrix(b, m, n);
        printf("C = A + B:\n");
        print_matrix(c, m, n);
        print_pcb(pcb);

        pcb.state = WAITING;
```

```
66          sleep(1);
67      }
68
69      return 0;
70  }
```

Process3.c

```
 1  #include <stdio.h>
 2  #include <string.h>
 3  #include <unistd.h>
 4
 5  #define READY 0
 6  #define RUNNING 1
 7  #define WAITING 2
 8
 9  struct PCB {
10      int process_id;
11      int priority;
12      int state;
13      int process_counter;
14  };
15
16  void print_pcb(struct PCB pcb)
17  {
18      printf("PCB: PID=%d, Priority=%d, State=%d, Counter=%d\n",
19             pcb.process_id, pcb.priority, pcb.state, pcb.process_counter);
20  }
21
22  int main(void)
23  {
24      struct PCB pcb = {3, 1, READY, 0};
25      char text[] = "operating system";
26      int count[256];
27      int i;
28
29      while (1) {
30          pcb.state = RUNNING;
31          pcb.process_counter++;
32
33          for (i = 0; i < 256; i++) {
34              count[i] = 0;
35          }
36
37          for (i = 0; text[i] != '\0'; i++) {
38              count[(unsigned char)text[i]]++;
39          }
40
41          printf("\n--- Process 3: Character frequency ---\n");
42          printf("String: \"%s\"\n", text);
43          printf("Frequency:\n");
44          for (i = 0; i < 256; i++) {
45              if (count[i] > 0) {
46                  if (i == ' ') {
47                      printf("'space': %d\n", count[i]);
48                  } else {
49                      printf("'%c': %d\n", i, count[i]);
50                  }
51              }
52          }
53          print_pcb(pcb);
54
55          pcb.state = WAITING;
56          sleep(1);
57      }
58
```

```
59      return 0;
60  }
```

Makefile

```
 1  CC = gcc
 2  CFLAGS = -Wall -Wextra -std=c11
 3
 4  all: lab3 process1 process2 process3
 5
 6  lab3: Lab3.c
 7    $(CC) $(CFLAGS) Lab3.c -o lab3
 8
 9  process1: Process1.c
10    $(CC) $(CFLAGS) Process1.c -o process1
11
12  process2: Process2.c
13    $(CC) $(CFLAGS) Process2.c -o process2
14
15  process3: Process3.c
16    $(CC) $(CFLAGS) Process3.c -o process3
17
18  clean:
19    rm -f lab3 process1 process2 process3
```

Хавсралт Б. Терминалын гаралтын зураг

Зураг 1. Lab3 нэгдсэн програм ажилласан terminal гаралт



